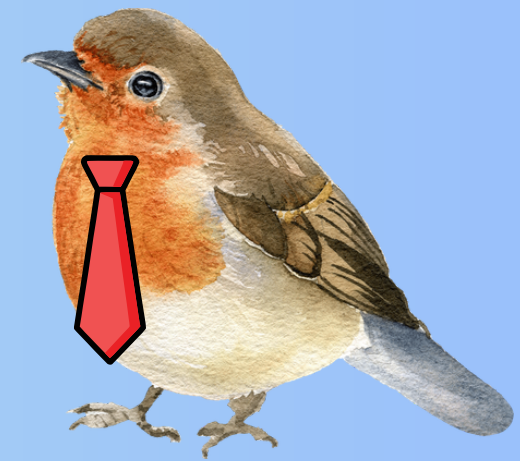


C# in Unity

DIG4540C

Why There Is No “main”

Austin Stanbury



Inversion of Control

Where is “main”?

Compare C# written as a console program vs. in Unity

```
// Console C#
static void Main() {
    var p = new Player();
    while (running) { p.Update(); }
}
```

```
// Unity
public class Player : MonoBehaviour {
    void Start() { }
    void Update() { }
}
```

In the consol, *you* own the loop - you define where the program starts and ends. You don't have this in Unity.

So how do things get called?

The call chain starts with MonoBehaviour

Unity keeps a list of every enabled MonoBehaviour in the scene and calls their lifecycle methods each frame.

However, scripts don't *need* to be MonoBehaviour to be executed in the scene. You can write classes, functions, etc. in plain C#, just like anywhere else. The only thing to know is that those scripts must be called at some point by a script that is registered as MonoBehaviour, on an active game object in the scene

The Lifecycle

Which lifecycle functions exist under MonoBehaviour

We have access to several lifecycle functions, let's check them out:

```
using UnityEngine;

public class LifecycleProbe : MonoBehaviour
{
    void Awake()          { Debug.Log($"{name}: Awake"); }
    void OnEnable()       { Debug.Log($"{name}: OnEnable"); }
    void Start()          { Debug.Log($"{name}: Start"); }
    void Update()         { Debug.Log($"{name}: Update frame {Time.frameCount}"); }
    void FixedUpdate()    { Debug.Log($"{name}: FixedUpdate"); }
    void LateUpdate()     { Debug.Log($"{name}: LateUpdate"); }
    void OnDisable()      { Debug.Log($"{name}: OnDisable"); }
    void OnDestroy()      { Debug.Log($"{name}: OnDestroy"); }
}
```

A mental model

Let's think about each function

- **Awake:** "I exist." Set up yourself. Don't touch other objects.
- **OnEnable:** "I'm on." Fires every time the object is activated. Subscribe to things here.
- **Start:** "Everyone exists." Safe to find and talk to other objects.
- **Update:** "A frame is happening." Input, non-physics movement, per-frame logic.
- **FixedUpdate:** "A physics step is happening." Fixed timestep; may run 0, 1, or several times per frame. Rigidbody forces go here.
- **LateUpdate:** "Everyone's Update is done." Cameras that follow things, anything that must see the final positions.
- **OnDisable / OnDestroy:** mirror of OnEnable / Awake. Unsubscribe, clean up.

Lifecycle Lab

Watch Unity Call Your Code

Setup: Attach LifecycleProbe.cs to the cube. Open the Console window (Window → General → Console). Clear it.

1. Press Play.

Read the first ten lines. Write down the order the methods fire in.

Which ones fire once? Which ones keep firing?

2. Pause the game. Uncheck the active box at the top of the cube's Inspector.

What printed? Now check it again. What printed this time?

Before you look: did Awake or Start fire again? Why or why not?

3. With the game still running, delete the cube from the Hierarchy.

What are the last two lines in the Console?

4. Stop the game. Duplicate the cube (Ctrl/Cmd+D). Rename one to "A" and one to "B". Clear the Console and press Play.

Look at the order of the first six lines. Does A finish its whole startup before B begins? What does that tell you about when it's safe for one object to talk to another?

Update and Real World Time

Update actually tracks frames

Consider this script:

```
public class Mover : MonoBehaviour
{
    void Update()
    {
        transform.Translate(0.1f, 0, 0);
    }
}
```

What happens when we change the framerate?

```
void Awake() { Application.targetFrameRate = 30; }
```

Time.deltaTime

Tracking real-world time intervals

We can time routines running in update to real-world time intervals by using a specific function:

```
public class Mover : MonoBehaviour
{
    public float speed = 3f;    // units per second

    void Update()
    {
        transform.Translate(speed * Time.deltaTime, 0, 0);
    }
}
```